

Teaching a model to write Jac

Synthetic Python → Jac data and a small finetune probe

Ayush Madhav Kumar

University of Michigan ▪ Jaseci

Synthetic Python → Jac ▪ MLX LoRA on Apple Silicon ▪ Qwen3 Coder & Gemma 4

No real Jac corpus exists

- Models trained on Python have weak priors on Jac's **data spatial idioms**: walkers, nodes and edges instead of functions and classes.
- Stock 30B and 26B Gemma 4 and Qwen3 Coder produce **0% runnable Jac**. They have never seen the language.

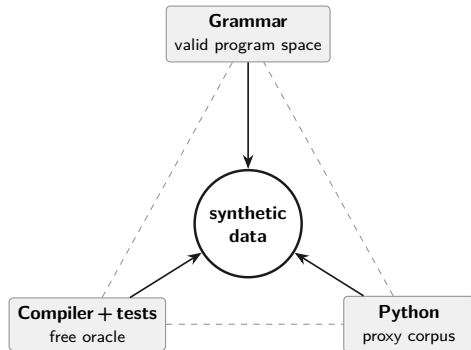
```
# a graph walk written as a loop
def traverse(graph, start):
    stack = [start]
    seen = set()
    while stack:
        n = stack.pop()
        seen.add(n)
        stack += graph[n]
    return seen
```

```
# the graph walks itself
walker Traverse {
    can step with Item entry {
        here.seen = True;
        visit [-->];
    }
}
node Item { has seen: bool = False; }
```

Three anchors for 100% synthetic data

1. **Grammar** is the distribution anchor. The Jac grammar defines the space of valid programs, and every sample is shaped to fit it.
2. **Compiler and tests** are a free oracle. Labelling is unlimited and costs nothing: reject anything that does not compile, run, or match its tests.
3. **Python** is the proxy distribution. Translate validated Python into Jac to borrow a large, correct source.

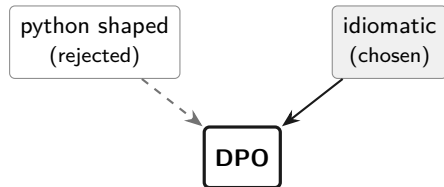
The quality gate is `jac run`: compile it, run it, and the output has to match. Not the type checker, which rejects plenty of code that runs.



the multi PLT method

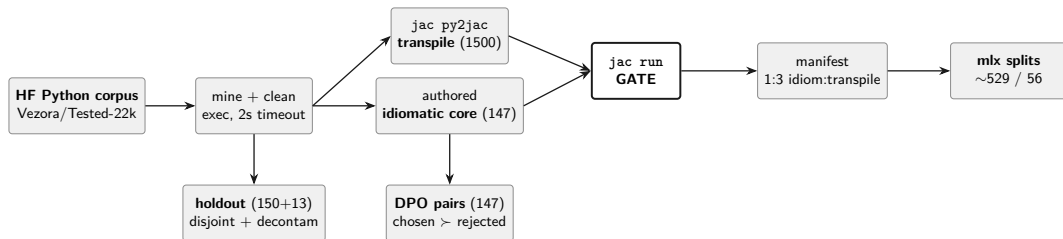
Data tiers, part 1: the three sources

1. **Idiomatic core** [147]. Jac written by hand or with an agent (Claude Code plus the Jac MCP): walkers, nodes, edges. *31 of these are graph shaped.*
2. **Transpile volume** [1500]. Python auto transpiled with jac py2jac and gated on behavior. Correct, but *Python shaped*. This is the cheap volume.
3. **DPO pairs** [147]. chosen is idiomatic, rejected is the transpile. This teaches the model *what is right versus what is wrong*.



a DPO pair is one task with
two answers (chosen $\succ$ rejected)

Data tiers, part 2: how the tiers are built



jac run gated ■ manifest **1:3** idiom:transpile ■ holdout decontaminated (disjoint offsets, 14 token shingles)

The data, tier by tier: real examples (1 of 2)

Tier 1 ■ Idiomatic core (authored: walker / node / edge)

task: product_of_tree, multiply values over a tree

```
node TNode { has val: int; }

walker Product {
  has result: int = 1;
  can mul with TNode entry {
    self.result *= here.val;
    visit [-->];      # the walk IS the program
  }
}

with entry {
  a = TNode(val=2); b = TNode(val=3); c = TNode(val=4);
  root ++> a;  a ++> b;  a ++> c;
  print((a spawn Product()).result);
}
```

Tier 2 ■ Transpile volume (jac py2jac, gated)

task: is_even, mechanical, Python shaped

```
def is_even(num: Any) -> object {
  return ((num % 2) == 0);
}

with entry {
  print(is_even(5));
  print(is_even(10));
}
```

Correct and runnable, but it is just Python with braces: Any and object types, no walker. The transpile gives cheap scale. The idiomatic core gives the shape.

The data, tier by tier: a real DPO pair (2 of 2)

task: count_children(graph, node), count a node's direct children

chosen idiomatic $\Rightarrow$ reward $\uparrow$

```
node GNode { has name: str; }

walker CountChildren {
  has count: int = 0;
  can chk with GNode entry {
    self.count = len(-->);
    disengage;
  }
}
```

rejected transpile, Python shaped $\Rightarrow$ reward $\downarrow$

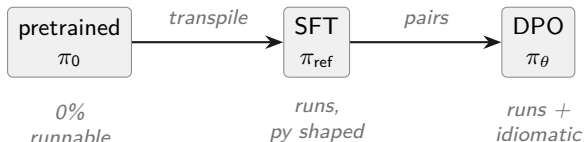
```
"""Return the number of direct
children of node."""
def count_children(graph: dict,
                    'node: str) -> int {
    return len(graph.get('node, []));
}
```

Same task, two correct answers. DPO trains **chosen** $\succ$ **rejected**, pushing generation off `graph.get(...)` and toward `len(-->)`.

What is SFT and DPO

- **SFT** is supervised finetuning. Imitate gold outputs, which teaches the model to *produce runnable Jac*.
- **DPO** is direct preference optimization. Learn from chosen $\succ$ rejected pairs, which *sharpens* the model toward idiom without a reward model.

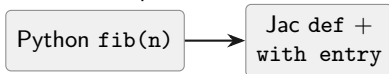
Here, SFT teaches it to *run*, and DPO pushes it from *runs* to *idiomatic*.



Function vs graph tasks

Function task

standalone pure function $\rightarrow$ Jac def



- idiomatic $\approx$ transpile $\Rightarrow$ **sim 0.97**
- **no idiom headroom**: a pure function legitimately needs no walker
- oracle: recorded test cases (150 holdout)

Graph task

dict and stack traversal $\rightarrow$ nodes / edges / walker

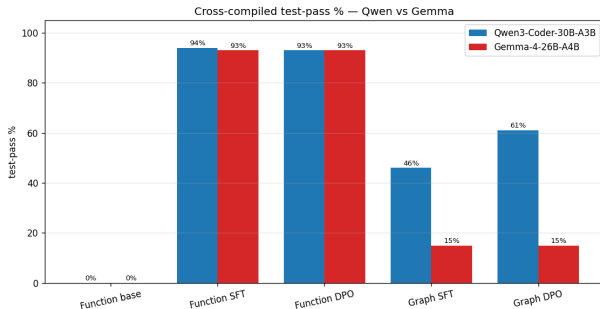


- idiomatic *diverges* hard $\Rightarrow$ **sim 0.26**, ~ 8 constructs
- **real idiom headroom**: this is where DPO can move the model
- one dict argument keeps the same eval harness (13 holdout)

The idiom axis only *exists* on graph tasks. On functions, “Python shaped” already *is* idiomatic, which is why DPO does nothing there and works here.

Graphs and stats: behavioral test pass

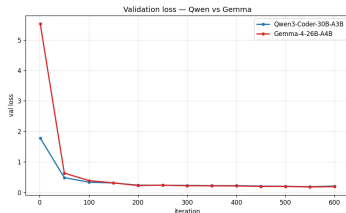
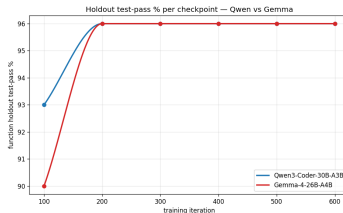
- Base eval from **cold start** (quantized Q4/Q8, run normally).
- **Base is 0%**: neither model knew Jac before this.
- Functions: both reach **~94%**. SFT and DPO tie, outputs stay Python shaped, and DPO does not move them.
- Graph: **Qwen pulls ahead** (46/61%) while Gemma stalls (15%).



Graphs and stats, part 2: training curves

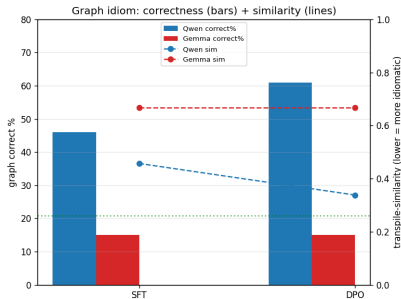
Validation loss, training loss, and the holdout learning curve are **nearly identical** for Qwen and Gemma. Same data, same config (LoRA r16, 600 iters, lr 2e-5).

The separation is *not* in loss. It is in *idiom* on graph tasks, which is the next slide.



Graph tier: where idiom has room to move

- On graph tasks idiomatic Jac *diverges* from Python, so the idiom axis is alive.
- **SFT** teaches the model to *run*. **DPO** pushes it the rest of the way to *idiomatic*.
- **Qwen**: SFT 46% → DPO **61%** correct; idiomatic share of correct 83% → **100%**; similarity 0.457 → **0.338** (ref 0.26).
- **Gemma**: graph SFT only 15% (~2/13, 0 constructs), so DPO has nothing to move. Idiom acquisition depends on the model.



What we proved

1. **Synthetic, compiler validated data teaches correct Jac.** 0% → **94%**
behavioral test pass on a decontaminated holdout.
2. **Idiom headroom plus DPO teaches idiomatic Jac.** On graph tasks Qwen3
Coder reaches **100%** idiomatic of correct outputs, up from 83% at SFT.
3. **The result is a basic Jac coding agent.** Roughly what Claude Code is, but for
Jac instead of Python.

100% synthetic data ■ a free compiler oracle ■ two MoE models that fit in 48 GB

Next up

- Mine a single codebase into an **RL loop**: how far is the resulting codebase from what you asked for?
- Is it unreasonable to train a model on *one* codebase?
- What **eval** fits that setting?

Shipped since: a live **training dashboard** (Monitor, Train, Ingest) that streams SFT and DPO runs in real time. It is the item that was “next” last time.

